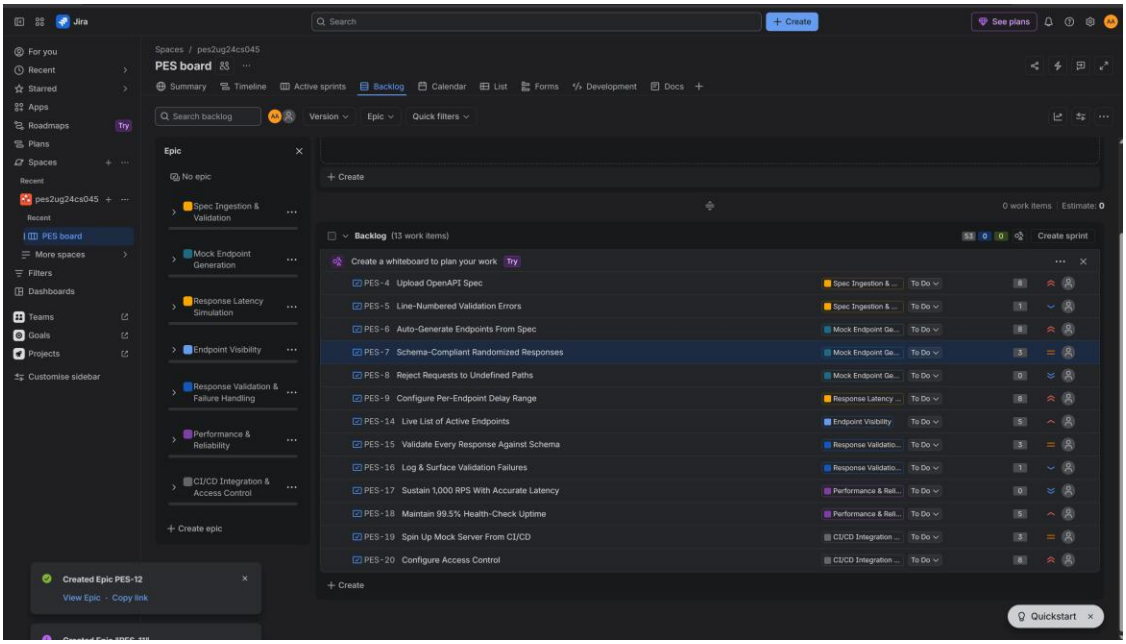
Lab 2: Agile Backlog Creation and Sprint Simulation, Reflection

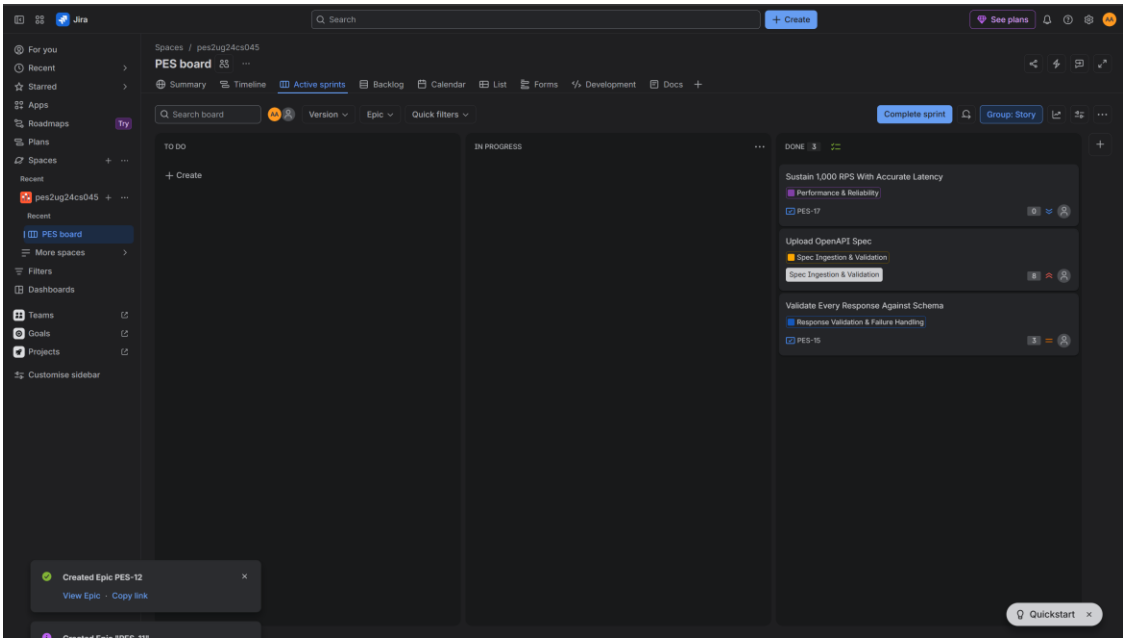
Akshansh Agrawal | SRN: PES2UG24CS045

System: OpenAPI Mock Server Generator (Problem Statement #41)

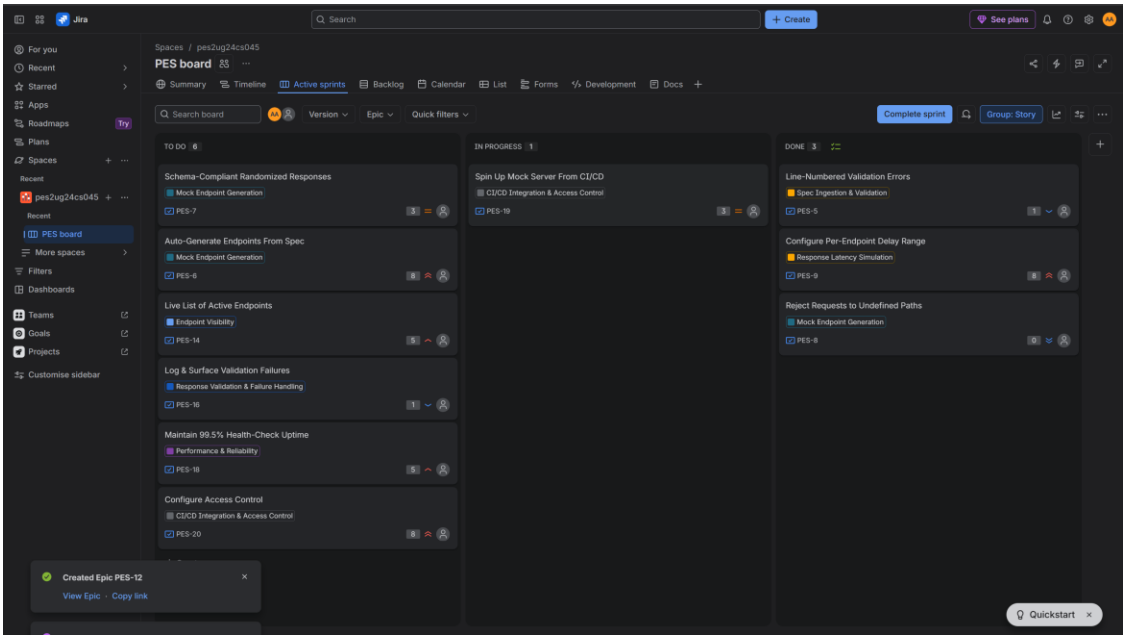
Backlog and story points



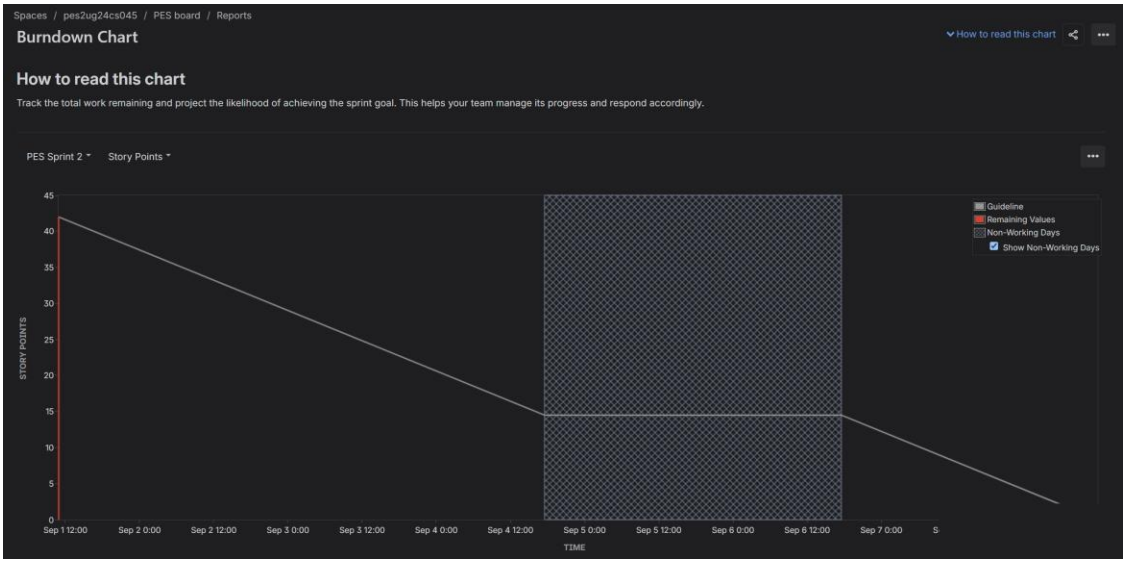
Sprint 1 board



Sprint 2 board



Burndown chart



1. Did your estimations reflect the actual effort?

Not entirely. Points across the backlog ranged from 0 to 8, but the spread didn't always track real complexity. “Reject Requests to Undefined Paths” and “Sustain 1,000 RPS With Accurate Latency” were both left at 0 points, even though the second one, load testing to 1,000 requests per second within plus or minus 10ms, is arguably one of the hardest items in the whole backlog. Meanwhile more routine items like “Upload OpenAPI Spec” and “Configure Access Control” were estimated at 8. This mismatch suggests the points were assigned quickly during setup rather than through real estimation discussion. In an actual team, these two 0 point items would need to be re-estimated before Sprint 2 planning.

2. Was your backlog well-prioritized?

The ordering by priority (High, then Medium, then Low) was reasonable, but the sprint split that came out of it wasn't balanced. Sprint 1 ended up with only 3 stories (Upload OpenAPI Spec, Validate Every Response Against Schema, Sustain 1,000 RPS), while Sprint 2 absorbed the remaining 10, about 42 story points on its own. In hindsight, the foundational high priority stories should have been spread more evenly across both sprints instead of compressing most of the backlog into Sprint 2.

3. How did your simulated sprint align with your plan?

Sprint 1 finished cleanly. All 3 committed stories reached Done. Sprint 2 told a different story: at the point captured in the board screenshot, 6 stories were still in To Do, 1 was In Progress, and only 3 were Done, out of 10 total. That's a clear sign Sprint 2 was overcommitted relative to the pace Sprint 1 demonstrated. A team that clears 3 stories comfortably in one sprint shouldn't be handed 10 in the next.

4. What insights did the burndown chart give about your team's capacity?

The Sprint 2 burndown starts at 42 points, which matches the sum of everything pulled into that sprint, and the guideline slopes down toward the sprint end, with the weekend correctly greyed out as non-working days. The actual “remaining work” line only shows the starting value so far, since the sprint is still mid-flight in the board screenshot and most items are still in To Do rather than Done. That's still a useful capacity signal: if the actual line stays flat near 42 for much longer, it points to the same overcommitment seen in the sprint board, too much pulled in relative to what the team demonstrated it could clear in Sprint 1. A team that burns down in a late rush right before the deadline, rather than a steady decline, usually means work was estimated or sequenced poorly, not that capacity was actually higher than planned.

Summary

Together, the two sprints point to an estimation problem more than a capacity problem. The team's real throughput, 3 completed stories in Sprint 1, was much lower than what got committed to Sprint 2, 10 stories and about 42 points. The fix for a real next sprint would be re-estimating the 0 point items and rebalancing the backlog so each sprint's total points stay closer to Sprint 1's demonstrated velocity.